

Podcast 24 — Observability

Podcast 24 — Observability

Length target: 15 min **Repo references:** `domain-1-organizational/24-cloudwatch.md`, `domain-3-continuous-improvement/03-x-ray.md`, `09-cloudwatch-synthetics-rum.md`, `10-managed-grafana-prometheus.md`, `07-aws-health-dashboard.md`

Topic & Scope

CloudWatch (metrics + logs + alarms), X-Ray (tracing), Synthetics (canaries), RUM (real user), Managed Grafana + AMP (Prometheus), Health Dashboard. The full observability stack.

Service Coverage Depth

Deep: - CloudWatch — metrics, custom metrics, alarms, composite alarms, Logs (Insights), dashboards - X-Ray — distributed tracing - Synthetics + RUM (frontend monitoring) - Managed Grafana + Managed Service for Prometheus

Brief: - AWS Health Dashboard + EventBridge integration - DevOps Guru (mention as ML-based ops anomaly) - ServiceLens (CloudWatch unified view)

Structured Outline

1. **Open (30s)** — “Three pillars of observability: metrics, logs, traces. Plus frontend monitoring and platform health. Six AWS services span them.”
2. **CloudWatch Metrics + Alarms (3.5 min)** — namespace + dimensions, standard / detailed monitoring, custom metrics (PutMetricData / EMF), alarm thresholds + actions, composite alarms (AND/OR multiple), anomaly detection
3. **CloudWatch Logs + Insights (3 min)** — log groups + streams, retention, encryption, Insights query language, metric filters, subscription filters (Kinesis / Lambda), Cross-Region copy
4. **X-Ray (2.5 min)** — distributed tracing, segments + subsegments, sampling rules, Service Map, OpenTelemetry via ADOT
5. **Synthetics + RUM (2.5 min)** — Synthetics canaries (scheduled probes from outside), RUM (JS snippet captures real user metrics), ServiceLens combines RUM + Synthetics + X-Ray
6. **Managed Grafana + AMP (1.5 min)** — Managed Grafana for dashboards across sources; Managed Prometheus for time-series metric store
7. **AWS Health Dashboard (30s)** — AWS-initiated events + EventBridge automation
8. **Rapid-Fire Trap Drill (60s)**

Must-Mention Exam Tips

- CloudWatch metrics: namespace + dimensions; standard (5-min) vs detailed (1-min, paid)

- High-resolution metrics: 1-second granularity for custom metrics
- Custom metrics via PutMetricData API or Embedded Metric Format (EMF) in Lambda
- Alarms: state (OK / ALARM / INSUFFICIENT_DATA), actions to SNS / Auto Scaling / EC2 actions / Systems Manager
- Composite alarm: combine multiple alarms with AND/OR logic
- Anomaly detection: ML-based dynamic thresholds
- CloudWatch Logs: log group → log stream → events; retention configurable per group
- Insights query language: SQL-like for log search/aggregation
- Metric filters: extract metrics from log patterns (e.g., count “ERROR” lines)
- Subscription filters: stream logs to Kinesis / Firehose / Lambda for real-time processing
- X-Ray: traces = end-to-end request; segments = service-level work; subsegments = downstream calls
- X-Ray sampling: default 1 trace/sec + 5% — tune via sampling rules
- OpenTelemetry via ADOT: vendor-neutral instrumentation; AWS-supported
- Synthetics canaries: Lambda + Puppeteer/Selenium, scheduled health probes
- RUM: JS snippet captures Core Web Vitals + JS errors + session paths, sends to CloudWatch
- ServiceLens: unified RUM + Synthetics + X-Ray + CloudWatch metrics + logs view
- Managed Grafana: SSO via Identity Center / SAML, no local users, \$9 editor / \$5 viewer per month
- Managed Prometheus (AMP): remote_write from ADOT collector, 150-day retention, PromQL queries
- AMP + Grafana + ADOT = EKS observability stack
- Health Dashboard: account events (issue / scheduledChange / accountNotification) via EventBridge

Must-Mention Exam Traps

- EXAM TRAP: CloudWatch metric basic granularity is 5 minutes — for 1-minute use Detailed Monitoring (paid)
- EXAM TRAP: CloudWatch alarms on missing data behavior — set “treat as missing” vs “breaching”
- EXAM TRAP: CloudWatch Logs retention defaults to “Never Expire” — set explicitly to control cost
- EXAM TRAP: Insights queries scan log data and bill per GB scanned
- EXAM TRAP: X-Ray default sampling is 1/sec + 5% — too low for low-traffic services
- EXAM TRAP: X-Ray trace retention is 30 days — for longer, export
- EXAM TRAP: Synthetics canaries cost per run — many regions × frequent intervals add up
- EXAM TRAP: RUM needs Cognito Identity Pool for unauthenticated browser uploads
- EXAM TRAP: Managed Grafana has NO local users — only IAM Identity Center or SAML
- EXAM TRAP: AMP doesn’t scrape — ADOT collector / Prometheus agent remote_writes to AMP
- EXAM TRAP: AMP uses Alertmanager — different alerting path from CloudWatch
- EXAM TRAP: Health Dashboard ≠ CloudWatch — Health is AWS-initiated events; CloudWatch is your resource metrics

- EXAM TRAP: CloudWatch agent ≠ X-Ray daemon — different agents, different responsibilities
- EXAM TRAP: CloudWatch Contributor Insights identifies top contributors to a metric (top-N analysis)
- EXAM TRAP: CloudWatch ServiceLens requires X-Ray + CloudWatch agent + RUM/Synthetics for full picture

Key Decision Matrix

Scenario	Pick
AWS service / EC2 metrics + alarms	CloudWatch
Centralized log collection + querying	CloudWatch Logs + Insights
Distributed tracing across microservices	X-Ray (or ADOT)
External availability probes	CloudWatch Synthetics canaries
Real-user page load + JS errors	CloudWatch RUM
PromQL queries on EKS metrics	Managed Service for Prometheus + ADOT
Dashboards across many data sources	Managed Grafana
React to AWS-side scheduled maintenance	Health Dashboard + EventBridge
ML-based ops anomaly detection	DevOps Guru
Top-N contributors to metric	CloudWatch Contributor Insights
Unified frontend-to-backend trace view	ServiceLens

Tone & Style

- Three pillars: metrics, logs, traces. Repeat throughout
- Distinguish “your resources” (CloudWatch) vs “AWS-side” (Health Dashboard)
- For Synthetics + RUM, frame as “scheduled probes vs real users”

Rapid-Fire Closer

“5-min default metric?” — “CloudWatch standard.” “1-min metric?” — “Detailed Monitoring.”
“Distributed traces?” — “X-Ray.” “Real user metrics?” — “RUM.” “AWS maintenance event?” —
“Health Dashboard + EventBridge.” “PromQL?” — “AMP.”